Funciones hash para verificar la integridad de la información, comparación entre MD5, SHA-1 y SHA-256

Trabajo de investigación

Analista en Sistemas de Computación

Licenciatura en Sistemas de Información

Inglés Técnico II

Profesora: Mgtr. Abraham Clara

Alumno: Dos Santos Axel Joan

Apóstoles, Misiones, Argentina

Año 2026

Abstract 2
1. Introducción 3
1.1 Planteamiento del problema 3
1.2 Pregunta de investigación 3
1.3 Objetivos 4
1.4 Justificación 4
1.5 Alcance y limitaciones 4
2. Marco teórico 5
2.1 La integridad en la seguridad de la información 5
2.2 Funciones hash 5
2.3 Aplicaciones 5
2.4 Principales algoritmos 6
2.5 Colisiones y caída de MD5 y SHA-1 6
2.6 Contraseñas, ataques de diccionario y sal 6
2.7 Antecedentes (Estado del Arte) 6
3. Metodología 7
3.1 Tipo de investigación 7
3.2 Fases 7
3.3 Entorno utilizado 7
3.4 Experimentos 8
3.5 Consideraciones éticas 9
4. Resultados 9
4.1 Experimento 1: efecto avalancha 9
4.2 Experimento 2: verificación de integridad 9
4.3 Experimento 3: longitud de salida y velocidad 10
4.4 Experimento 4: ataque de diccionario 10
4.5 Experimento 5: colisiones con hashes truncados 11
5. Discusión 12
6. Conclusiones 13
7. Recomendaciones 14
8. Referencias 15
9. Anexos 17
Anexo A. Resumen del procedimiento 17
Anexo B. Código fuente 17
Anexo C. Resultados guardados 17

Resumen

La integridad de la información (que los datos no se modifiquen) es uno de los pilares de la seguridad informática. Las funciones hash sirven para comprobar si un archivo fue alterado, ya sea por error o a propósito. Sin embargo, no todos los algoritmos son igual de seguros: MD5 y SHA-1 tienen colisiones demostradas y aun así se siguen usando.

En este trabajo se explica cómo funcionan los hashes y por qué MD5 y SHA-1 dejaron de ser seguros, y se los compara con SHA-256 y SHA-512. Para eso se hicieron cinco experimentos en Python con la biblioteca `hashlib`: efecto avalancha, verificación de un archivo, comparación de velocidad, ataque de diccionario a contraseñas (con y sin sal) y colisiones en hashes truncados.

Resultados: los cuatro algoritmos cambian cerca del 50 % de sus bits ante un cambio mínimo, y todos detectaron la modificación de un solo bit en un archivo. SHA-256 fue el más rápido (1 615 MB/s) y MD5 el más lento (575 MB/s). En el ataque de diccionario se recuperaron 6 de 8 contraseñas, pero la sal casi triplicó el trabajo del atacante. PBKDF2 resultó unas 40 000 veces más lento que SHA-256 simple.

Conclusiones: MD5 y SHA-1 no son seguros para usos criptográficos. SHA-256 es una buena opción para verificar integridad. Para guardar contraseñas conviene usar sal y algoritmos lentos como PBKDF2, bcrypt o Argon2.

Palabras clave: funciones hash, integridad, MD5, SHA-1, SHA-256, colisiones, sal.

Abstract

Information integrity (meaning that data is not modified) is one of the pillars of information security. Hash functions are used to check whether a file has been altered, either by mistake or on purpose. However, not all algorithms are equally secure: MD5 and SHA-1 have proven collisions and are still in use.

This work explains how hashes work and why MD5 and SHA-1 stopped being secure, and compares them with SHA-256 and SHA-512. To do this, five experiments were carried out in Python using the hashlib library: avalanche effect, file verification, speed comparison, a dictionary attack on passwords (with and without salt), and collisions on truncated hashes.

Results: all four algorithms change about 50% of their bits after a minimal change, and all of them detected the modification of a single bit in a file. SHA-256 was the fastest (1,615 MB/s) and MD5 the slowest (575 MB/s). In the dictionary attack, 6 out of 8 passwords were recovered, but salt almost tripled the attacker's work. PBKDF2 was about 40,000 times slower than plain SHA-256.

Conclusions: MD5 and SHA-1 are not secure for cryptographic uses. SHA-256 is a good option for verifying integrity. To store passwords, it is advisable to use salt and slow algorithms such as PBKDF2, bcrypt or Argon2.

Keywords: hash functions, integrity, MD5, SHA-1, SHA-256, collisions, salt.

1. Introducción

Casi toda actividad informática depende de que los datos lleguen y se mantengan tal como fueron creados: una actualización, un documento, un respaldo o una contraseña guardada. A esta propiedad se la llama **integridad**, y junto con la confidencialidad y la disponibilidad forma la base de la seguridad de la información. Las funciones hash son una de las herramientas más usadas para protegerla.

Distintos autores han estudiado su evolución y su seguridad (Khan et al., 2022; Maetouq et al., 2018) y su uso práctico para verificar archivos y certificados (Anwar et al., 2021; Boyanov, 2024). Este trabajo busca explicar cómo funcionan los hashes, por qué MD5 y SHA-1 dejaron de ser seguros y cómo comprobarlo con herramientas simples.

1.1 Planteamiento del problema

Cuando se descarga un archivo de internet no hay garantía de que sea idéntico al original: puede haber errores de transmisión o alguien podría haberlo reemplazado por una versión maliciosa. Por eso muchos sitios publican el hash del archivo para poder compararlo.

El problema es que no todos los algoritmos son seguros. MD5 y SHA-1 fueron estándares durante años, pero hoy se sabe que se pueden fabricar dos archivos distintos con el mismo hash (Jasek, 2015). Aun así, siguen apareciendo en sitios y sistemas antiguos.

1.2 Pregunta de investigación

Pregunta principal: ¿Hasta qué punto los hashes permiten verificar la integridad de archivos y datos, y por qué MD5 y SHA-1 ya no son recomendables frente a SHA-256?

Preguntas secundarias:

1. ¿Qué propiedades debe cumplir un hash seguro?
2. ¿Qué pasó con MD5 y SHA-1?
3. ¿Cómo puede un usuario comprobar la integridad de un archivo?
4. ¿Qué papel cumple la sal al guardar contraseñas?

1.3 Objetivos

Objetivo general: analizar cómo funcionan y qué tan seguras son las funciones hash para verificar integridad, mostrando con experimentos sencillos las diferencias entre algoritmos antiguos y actuales.

Objetivos específicos:

1. Describir las propiedades principales de un hash.
2. Explicar brevemente la caída de MD5 y SHA-1.
3. Mostrar con Python el efecto avalancha y la verificación de un archivo.
4. Comparar MD5, SHA-1, SHA-256 y SHA-512 en longitud de salida y velocidad.
5. Mostrar cómo un ataque de diccionario afecta a las contraseñas y cómo ayuda la sal.
6. Dar recomendaciones de uso según el caso.

1.4 Justificación

Comparar dos cadenas de texto permite detectar desde una descarga dañada hasta una manipulación maliciosa, lo que es barato y muy útil. Pero solo sirve si se elige un buen algoritmo. Entender por qué MD5 y SHA-1 fueron abandonados, y poder comprobarlo con herramientas al alcance de cualquier estudiante, ayuda a tomar decisiones técnicas basadas en evidencia y no en la costumbre.

1.5 Alcance y limitaciones

Alcance: se estudian MD5, SHA-1, SHA-256 y SHA-512, su historia de seguridad y su uso en verificación de archivos y contraseñas.

Limitaciones:

- No se hace un análisis matemático profundo.
- Los experimentos se hicieron en un solo equipo, por lo que los tiempos dependen de ese hardware.
- Las contraseñas usadas son inventadas; no se usaron datos reales.
- No se generaron colisiones reales de MD5 o SHA-1: se usaron hashes truncados y material público ya documentado.

2. Marco teórico

2.1 La integridad en la seguridad de la información

La seguridad de la información se explica con tres pilares: **confidencialidad** (solo accede quién está autorizado), **integridad** (los datos no se alteran) y **disponibilidad** (la información se puede usar cuando se necesita). Los mecanismos de integridad no ocultan la información; permiten detectar si cambió.

2.2 Funciones hash

Una función hash recibe datos de cualquier tamaño y devuelve un resultado de tamaño fijo, llamado *hash* o *digest*. Por ejemplo, SHA-256 siempre devuelve 256 bits (64 caracteres hexadecimales), sin importar si la entrada es una palabra o un archivo enorme. Un hash criptográfico debe cumplir:

- **Determinismo:** la misma entrada da siempre el mismo resultado.
- **Efecto avalancha:** un cambio mínimo en la entrada cambia el resultado por completo.
- **Resistencia a la preimagen:** Teniendo el hash resulta computacionalmente inviable obtener la entrada.
- **Resistencia a la segunda preimagen:** dada una entrada, resulta computacionalmente inviable obtener otra con el mismo hash.
- **Resistencia a colisiones:** Es computacionalmente inviable obtener dos entradas distintas con el mismo hash.

El hash no es lo mismo que el cifrado: el cifrado se puede revertir con una clave, mientras que el hash es de un solo sentido.

2.3 Aplicaciones

- **Verificar archivos:** se compara el hash publicado con el calculado localmente para comprobar la integridad del archivo.
- **Guardar contraseñas:** Es utilizado cuando se desea guardar contraseñas en base de datos por ejemplo, en donde en lugar de almacenar directamente la contraseña se almacena su código hash, para luego comparar tal código con el hash de la contraseña introducida por el usuario, de este modo aunque la base de datos sea vulnerada, obtener la contraseña a partir del hash resultará cuando menos tedioso o inviable.
- **Firmas digitales y certificados:** se firma el hash del documento (Anwar et al., 2021).
- **Detectar cambios en textos sensibles** (Almazrooie et al., 2020).
- **Git y blockchain:** usan hashes para identificar y enlazar su contenido.

2.4 Principales algoritmos

Algoritmo	Salida	Estado actual
MD5	128 bits	Obsoleto: se generan colisiones con facilidad.
SHA-1	160 bits	Obsoleto: en 2017 se demostró una colisión real (SHAttered).
SHA-256 (SHA-2)	256 bits	Seguro y muy usado.
SHA-512 (SHA-2)	512 bits	Seguro.

Tabla 1. Algoritmos hash y su estado de seguridad.

2.5 Colisiones y caída de MD5 y SHA-1

Una colisión ocurre cuando dos entradas distintas dan el mismo hash. Siempre existen, porque hay infinitas entradas y una cantidad limitada de salidas; la seguridad depende de que encontrarlas sea inviable. En MD5, desde mediados de los años 2000 se publicaron métodos para generar colisiones rápidamente (Jasek, 2015). SHA-1 resistió más, pero en 2017 el ataque **SHAttered** mostró dos PDF distintos con el mismo SHA-1 (Stevens et al., 2017). Esto permite que un atacante prepare un archivo legítimo y otro malicioso con la misma huella, lo que debilita las firmas y certificados.

2.6 Contraseñas, ataques de diccionario y sal

Si las contraseñas se guardan con un hash simple, un atacante que obtenga la base de datos puede calcular los hashes de contraseñas comunes y compararlos (ataque de diccionario). La **sal** es un valor aleatorio distinto para cada usuario que se agrega a la contraseña antes de aplicar el hash: así, contraseñas iguales dan hashes distintos y las tablas precalculadas dejan de servir. Además, usar algoritmos lentos (PBKDF2, bcrypt, Argon2) hace más costoso cada intento.

2.7 Antecedentes (Estado del Arte)

Trabajo	Aporte para este estudio
Jasek (2015)	Seguridad de MD5 y SHA-1 y ataques que los debilitaron.
Maetouq et al. (2018)	Comparación de algoritmos hash frente a ataques.

Trabajo	Aporte para este estudio
Almazrooie et al. (2020)	Verificación de integridad de textos digitales con hash.
Anwar et al. (2021)	Uso de hashes para verificar datos de certificados.
Khan et al. (2022)	Evolución y análisis de la familia SHA.
Cherckesova et al. (2022)	Propuesta de un nuevo hash resistente a colisiones.
Boyanov (2024)	Uso práctico de MD5, SHA-1 y SHA-256 para verificar archivos.
Davydov et al. (2025)	Análisis de la fortaleza de SHA-256.

Tabla 2. Antecedentes revisados.

3. Materiales y métodos

3.1 Tipo de investigación

Investigación **exploratoria y descriptiva**, con una parte **experimental** sencilla. Es mixta: cualitativa en la revisión de información y cuantitativa en la medición de resultados (bits cambiados, tiempos, intentos).

3.2 Fases

1. **Revisión documental:** estándares (RFC 1321, FIPS 180-4), el caso SHAttered y artículos científicos.
2. **Experimentos:** cinco pruebas en Python.
3. **Análisis:** interpretación de resultados y recomendaciones.

3.3 Entorno utilizado

Elemento	Detalle
Lenguaje	Python 3.13.7
Biblioteca	hashlib (biblioteca estándar), además de os, time, shutil, statistics

Elemento	Detalle
Sistema operativo	Microsoft Windows 11 Home (versión 10.0.26200)
Procesador	AMD Ryzen 5 7535HS, 6 núcleos y 12 hilos
Memoria	8 GB de RAM
Editor	Antigravity IDE

3.4 Experimentos

N.º	Experimento	Qué se mide
1	Efecto avalancha	Cantidad y porcentaje de bits que cambian al modificar la entrada
2	Verificación de integridad	Si el hash coincide tras modificar el archivo
3	Comparación de algoritmos	Longitud de salida y tiempo de cálculo
4	Ataque de diccionario	Contraseñas recuperadas y cálculos de hash necesarios
5	Colisiones con hashes truncados	Intentos hasta encontrar una colisión

- **Experimento 1:** se calculó el hash de un texto y de una versión con un cambio mínimo, y se contaron los bits distintos. Se espera cerca del 50 %.
- **Experimento 2:** se calculó el hash de un archivo, se modificó una copia y se volvió a calcular.
- **Experimento 3:** se midió el tiempo que tarda cada algoritmo en procesar 10 MB de datos, repetido 50 veces (se usa la mediana).
- **Experimento 4:** se creó una base de usuarios ficticios y se simuló un atacante con un diccionario de contraseñas comunes, con hash simple y con sal. También se comparó SHA-256 con PBKDF2.
- **Experimento 5:** se tomaron solo los primeros n bits de un hash (16, 20, 24 y 32) y se buscó una colisión. Además, se verificaron los PDF públicos de SHAttered.

El código y los pasos de cada experimento se resumen en el Anexo A.

3.5 Consideraciones éticas

Las contraseñas y usuarios son ficticios, no se atacó ningún sistema real y los archivos usados son propios o de distribución pública.

4. Resultados

4.1 Experimento 1: efecto avalancha

Se usó el texto "Hola mundo" y dos cambios mínimos: (A) 'm' por 'M' y (B) agregar un punto al final.

Algoritmo	Bits totales	Prueba A: bits que cambiaron	Prueba B: bits que cambiaron
MD5	128	66 (51,56 %)	62 (48,44 %)
SHA-1	160	79 (49,38 %)	90 (56,25 %)
SHA-256	256	136 (53,12 %)	133 (51,95 %)
SHA-512	512	272 (53,12 %)	249 (48,63 %)

Tabla 3. Efecto avalancha.

En todos los casos cambió cerca del 50 % de los bits (entre 48,44 % y 56,25 %), lo que indica un buen efecto avalancha.

4.2 Experimento 2: verificación de integridad

Se creó un archivo de 355 bytes y se calculó su hash. Luego se cambió un solo bit en una copia (byte en la posición 10, 20 → 21) y se recalculó.

Algoritmo	Hash original	Hash modificado	¿Coincide?
MD5	b57566c0...	f694c7aa...	No
SHA-1	9b354d12...	47342697...	No
SHA-256	8d55e9e1...	557ca1b3...	No
SHA-512	2dfddb5a...	9a01efe9...	No

Tabla 4. Verificación de integridad.

Los cuatro algoritmos detectaron el cambio de un solo bit.

4.3 Experimento 3: longitud de salida y velocidad

Se procesaron 10 MB de datos aleatorios, 50 veces por algoritmo (mediana).

Algoritmo	Salida (bits)	Caracteres hex	Tiempo mediano (s)	Velocidad (MB/s)
MD5	128	32	0,017395	574,88
SHA-1	160	40	0,006332	1 579,32
SHA-256	256	64	0,006189	1 615,67
SHA-512	512	128	0,014533	688,07

Tabla 5. Longitud de salida y rendimiento.

SHA-256 fue el más rápido (1 615 MB/s), seguido por SHA-1 (1 579 MB/s). MD5, aunque tiene la salida más corta, fue el más lento (575 MB/s).

4.4 Experimento 4: ataque de diccionario

Se creó una base de 8 usuarios ficticios: 6 con contraseñas comunes (123456, password, qwerty, letmein, admin, iloveyou) y 2 con contraseñas fuertes (S3gur4!2026, Xk9$mPz!wQ). El diccionario del atacante tenía 50 contraseñas frecuentes y se usó SHA-256.

Escenario	Contraseñas recuperadas	Cálculos de hash
Hash simple (sin sal)	6 de 8 (75 %)	50
Hash con sal	6 de 8 (75 %)	141

Tabla 6. Resultados del ataque de diccionario.

Sin sal, el atacante calcula los 50 hashes una sola vez y los compara con todos los usuarios. Con sal, debe recalcularlos para cada usuario, por lo que necesitó casi el triple de cálculos (141). Las dos contraseñas fuertes no fueron recuperadas en ningún caso.

Algoritmo	Tiempo por cálculo (s)
SHA-256 simple	0,00000131

Algoritmo	Tiempo por cálculo (s)
PBKDF2 (100 000 iteraciones)	0,052245

Tabla 7. Hash rápido vs. hash lento.

PBKDF2 resultó unas **40 000 veces más lento** que SHA-256 simple, lo que hace mucho más costoso probar contraseñas.

4.5 Experimento 5: colisiones con hashes truncados

Se truncó SHA-256 a 16, 20, 24 y 32 bits y se buscó una colisión (5 repeticiones, mediana).

Bits	Intentos teóricos (2^(n/2))	Mediana obtenida
16	~256	298
20	~1 024	896
24	~4 096	5 166
32	~65 536	65 188

Tabla 8. Intentos necesarios para hallar una colisión.

Los resultados coinciden con la teoría: los intentos crecen con la raíz cuadrada del espacio de salida (paradoja del cumpleaños). Al pasar de 16 a 32 bits, los intentos aumentaron unas 219 veces (la teoría predice 256), por lo que más bits de salida dan mucha más resistencia a colisiones.

Archivos SHAttered:

Archivo	SHA-1	SHA-256
shattered-1.pdf	38762cf7f55934b34d179ae6 a4c80cadccbb7f0a	2bb787a73e37352f...
shattered-2.pdf	38762cf7f55934b34d179ae6 a4c80cadccbb7f0a	d4488775d29bdef7...

Tabla 9. PDF del ataque SHAttered (hashes publicados en shattered.io).

Los dos PDF son distintos pero tienen el mismo SHA-1 (una colisión real; Stevens et al., 2017). Sus SHA-256 son diferentes, por lo que SHA-256 sí los distingue.

5. Discusión

Efecto avalancha. Los cuatro algoritmos, incluso MD5 y SHA-1, cambiaron cerca del 50 % de sus bits, como se espera de un buen hash (Khan et al., 2022). Pero esto por sí solo no garantiza seguridad: MD5 tiene buen efecto avalancha y aun así tiene colisiones.

Integridad de archivos. Cambiar un solo bit produjo un hash totalmente distinto en todos los algoritmos. Esto confirma que los hashes sirven para verificar integridad (Boyanov, 2024; Anwar et al., 2021), y por eso conviene que los sitios publiquen el hash junto al archivo.

Rendimiento. SHA-256 fue más rápido que MD5 en este equipo. Una posible explicación es que el procesador y la biblioteca usada tienen optimizaciones para SHA-256. Así, MD5 ya no tiene la ventaja de ser "más rápido", y no hay razón para preferirlo.

Contraseñas y sal. El 75 % de las contraseñas fueron recuperadas porque eran muy comunes. Se pueden sacar tres ideas: (1) la sal no evita que se descubran contraseñas débiles, pero aumenta el trabajo del atacante (de 50 a 141 cálculos) y deja inútiles las tablas precalculadas; (2) las contraseñas fuertes resisten en ambos casos; (3) PBKDF2 es unas 40 000 veces más lento que SHA-256, lo que limita mucho los intentos por segundo de un atacante.

Paradoja del cumpleaños. Los intentos necesarios para encontrar una colisión crecen según 2^(n/2). Para MD5 (128 bits) eso son 2^64 operaciones, algo que ya se puede alcanzar con hardware actual (Jasek, 2015). Para SHA-256 son 2^128, un valor que se considera seguro (Davydov et al., 2025).

Caso SHAttered. La colisión real de SHA-1 confirma que ese algoritmo no es confiable, mientras que SHA-256 distingue correctamente ambos archivos.

Relación con otros estudios. Los resultados coinciden con la literatura: Maetouq et al. (2018) señalan que SHA-2 resiste mejor las colisiones, y Cherckesova et al. (2022) proponen nuevos algoritmos para superar las limitaciones de la familia MD/SHA-1.

6. Conclusiones

Pregunta principal. Las funciones hash permiten verificar bien la integridad de archivos y datos: el experimento 2 mostró que cambiar un solo bit produce un hash totalmente distinto. Pero la confianza depende del algoritmo. MD5 y SHA-1 conservan el efecto avalancha, pero ya no son recomendables porque sus colisiones están demostradas (experimento 5 y caso SHAttered). SHA-256 es más seguro y, en las pruebas, incluso más rápido que MD5.

Preguntas secundarias:

1. **Propiedades de un hash seguro:** determinismo, efecto avalancha y resistencia a preimagen, segunda preimagen y colisiones. El efecto avalancha se cumplió en los cuatro algoritmos, pero la resistencia a colisiones está comprometida en MD5 y SHA-1.
2. **Qué pasó con MD5 y SHA-1:** MD5 fue roto desde mediados de los años 2000 y SHA-1 en 2017 con SHAttered, por lo que no deben usarse en firmas ni certificados.
3. **Cómo verificar un archivo:** calculando su hash con una herramienta del sistema (Get-FileHash en Windows, sha256sum en Linux) o con Python, y comparándolo con el valor publicado.
4. **Papel de la sal:** obliga al atacante a calcular los hashes uno por uno para cada usuario (en el experimento, de 50 a 141 cálculos) y evita las tablas precalculadas. Pero no protege contraseñas débiles; hay que combinarla con algoritmos lentos como PBKDF2, bcrypt o Argon2.

Objetivos específicos: todos se cumplieron. Se describieron las propiedades de los hashes, se explicó la caída de MD5 y SHA-1, se hicieron los experimentos en Python, se compararon los cuatro algoritmos y se dieron recomendaciones (sección 7).

Conclusión general: elegir un algoritmo hash no es una decisión menor. MD5 y SHA-1 deben considerarse obsoletos para seguridad. SHA-256 es hoy una opción confiable y eficiente. Para contraseñas, un hash rápido no alcanza: se deben usar funciones lentas diseñadas para ese fin.

7. Recomendaciones

Para verificar archivos y descargas:

- Usar **SHA-256**.
- Si un sitio solo publica MD5 o SHA-1, tomarlo como una verificación débil: sirve para detectar errores de descarga, pero no manipulaciones maliciosas.
- Verificar siempre el hash con Get-FileHash (PowerShell) o sha256sum (Linux/macOS).

Para guardar contraseñas:

- No usar un hash simple (SHA-256, MD5, etc.), ni siquiera con sal.
- Usar **Argon2**, **bcrypt** o **PBKDF2** (con al menos 100 000 iteraciones).
- Generar una sal aleatoria distinta para cada usuario (mínimo 16 bytes) y guardarla junto al hash.
- Pedir contraseñas largas y complejas, porque ninguna técnica protege contraseñas triviales.

Para firmas y certificados:

- No aceptar certificados firmados con MD5 o SHA-1.
- Usar SHA-256 o superior.

Para administradores y desarrolladores:

- Revisar sistemas antiguos que usen MD5 o SHA-1 en funciones de seguridad y planificar su reemplazo.
- MD5 solo puede mantenerse en usos sin riesgo de seguridad, como detectar archivos duplicados.

Para el ámbito educativo:

- Los experimentos de este trabajo se pueden repetir con la biblioteca estándar de Python, sin instalar nada.

8. Referencias

Almazrooie, M., Samsudin, A., Gutub, A. A.-A., Salleh, M. S., Omar, M. A., & Hassan, S. A. (2020). Integrity verification for digital Holy Quran verses using cryptographic hash function and compression. *Journal of King Saud University – Computer and Information Sciences, 32*(1), 24–34. https://doi.org/10.1016/j.jksuci.2018.02.006

Anwar, M. R., Apriani, D., & Adianita, I. R. (2021). Hash algorithm in verification of certificate data integrity and security. *Aptisi Transactions on Technopreneurship (ATT), 3*(2), 67–74. https://doi.org/10.34306/att.v3i2.212

Boyanov, P. K. (2024). Practical applications of hash functions MD5, SHA-1, and SHA-256 using various software tools to verify the integrity of files. *Journal Scientific and Applied Research, 27*. (DOI y enlace: a completar)

Cherckesova, L. V., Safaryan, O. A., Lyashenko, N. G., & Korochentsev, D. A. (2022). Developing a new collision-resistant hashing algorithm. *Mathematics, 10*(15), Article 2769. https://doi.org/10.3390/math10152769

Davydov, V. V., Pikhtovnikov, M. D., Kiryanova, A. P., & Zaikin, O. S. (2025). Analysis of the cryptographic strength of the SHA-256 hash function using the SAT approach. *Scientific and Technical Journal of Information Technologies, Mechanics and Optics, 25*(3), 428–437. https://doi.org/10.17586/2226-1494-2025-25-3-428-437

Jasek, R. (2015). SHA-1 and MD5 cryptographic hash functions: Security overview. *Communications – Scientific Letters of the University of Zilina, 17*(1), 73–80. https://doi.org/10.26552/com.C.2015.1.73-80

Khan, B. U. I., Olanrewaju, R. F., Morshidi, M. A., Mir, R. N., Kiah, M. L. B. M., & Khan, A. M. (2022). Evolution and analysis of secure hash algorithm (SHA) family. *Malaysian Journal of Computer Science, 35*(3). (DOI: a verificar)

Maetouq, A., Daud, S. M., Ahmad, N. A., Maarop, N., Sjarif, N. N. A., & Abas, H. (2018). *Comparison of hash function algorithms against attacks: A review*. Universiti Teknologi Malaysia. (Datos de publicación y enlace: a completar)

National Institute of Standards and Technology. (2015). *Secure Hash Standard (SHS)* (FIPS PUB 180-4). U.S. Department of Commerce. https://doi.org/10.6028/NIST.FIPS.180-4

Rivest, R. (1992). *The MD5 message-digest algorithm* (RFC 1321). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc1321

Stevens, M., Bursztein, E., Karpman, P., Albertini, A., & Markov, Y. (2017). The first collision for full SHA-1. En J. Katz & H. Shacham (Eds.), *Advances in Cryptology – CRYPTO 2017* (pp. 570–596). Springer. https://doi.org/10.1007/978-3-319-63688-7_19

9. Anexos

Anexo A. Resumen del procedimiento

- **Experimento 1:** se calcularon los hashes de "Hola mundo" y de dos variantes ("Hola Mundo" y "Hola mundo.") con MD5, SHA-1, SHA-256 y SHA-512. Se pasaron a binario y se contaron los bits distintos.
- **Experimento 2:** se creó un archivo de 355 bytes, se calcularon sus hashes, se copió el archivo cambiando un solo bit y se recalcularon.
- **Experimento 3:** se generaron 10 MB de datos aleatorios con os.urandom() y se midió el tiempo de cada algoritmo 50 veces con time.perf_counter() (se reporta la mediana).
- **Experimento 4:** se creó una base de 8 usuarios ficticios (6 con contraseñas débiles y 2 fuertes) y se guardaron con SHA-256 sin sal y con sal de 16 bytes. Se simuló un atacante con 50 contraseñas comunes y se comparó PBKDF2 (100 000 iteraciones) con SHA-256 simple.
- **Experimento 5:** se truncaron hashes SHA-256 a 16, 20, 24 y 32 bits y se buscó una colisión (5 repeticiones por tamaño). También se verificaron los hashes públicos de los PDF de SHAttered.

Anexo B. Código fuente

- experimento1_avalancha.py
- experimento2_integridad.py
- experimento3_comparacion.py
- experimento4_diccionario.py
- experimento5_birthday.py

Todos usan solo la biblioteca estándar de Python (hashlib, os, time, statistics, json, shutil).

Anexo C. Resultados guardados

Cada script genera un archivo JSON con los datos completos: resultados_exp1.json, resultados_exp2.json, resultados_exp3.json, resultados_exp4.json y resultados_exp5.json.